Practical 4

In [13]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, r2_score
```

In [14]:
```
df=pd.read_csv("BostonHousing.csv")
```

In [15]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype  
---  ------   --------------  -----  
 0   crim     506 non-null    float64
 1   zn       506 non-null    float64
 2   indus    506 non-null    float64
 3   chas     506 non-null    int64  
 4   nox      506 non-null    float64
 5   rm       501 non-null    float64
 6   age      506 non-null    float64
 7   dis      506 non-null    float64
 8   rad      506 non-null    int64  
 9   tax      506 non-null    int64  
 10  ptratio  506 non-null    float64
 11  b        506 non-null    float64
 12  lstat    506 non-null    float64
 13  medv     506 non-null    float64
dtypes: float64(11), int64(3)
memory usage: 55.5 KB
```

In [16]:
```
df.columns
```

Out[16]:
```
Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis', 'rad', 'tax',
       'ptratio', 'b', 'lstat', 'medv'],
      dtype='str')
```

In [17]:
```
df.isnull().sum()
```

Out[17]:
```
crim       0
zn         0
indus      0
chas       0
nox        0
rm         5
age        0
dis        0
rad        0
tax        0
ptratio    0
b          0
lstat      0
medv       0
dtype: int64
```

crim – Per capita crime rate in that town. It shows how much crime occurs in the area.

zn – Proportion of residential land zoned for large houses (lots over 25,000 sq.ft). It indicates how much land is reserved for big residential buildings.

indus – Proportion of non-retail business acres per town. It represents how industrial or commercial the area is.

chas – Charles River dummy variable. It is 1 if the house is near the Charles River and 0 if it is not.

nox – Nitric oxide concentration in the air. It represents the pollution level of the area.

rm – Average number of rooms per dwelling. Houses with more rooms usually have higher value.

age – Proportion of owner-occupied houses built before 1940. It indicates how old the houses in the area are.

dis – Weighted distance to five Boston employment centers. It shows how far the houses are from major workplaces.

rad – Index of accessibility to radial highways. It indicates how easily highways can be accessed from that area.

tax – Property tax rate per $10,000. It represents how much property tax people pay.

ptratio – Pupil-teacher ratio in the town. It reflects the education quality in that area.

b – A variable related to the proportion of Black population used in the original dataset formula.

lstat – Percentage of lower-status population in the area.

medv – Median value of owner-occupied homes in thousands of dollars. This is the dependent variable because the model tries to predict house price using the other variables.

In [18]:
```
df["rm"]=df["rm"].fillna(df["rm"].mean())
```

In [19]:
```
df
```

Out[19]:

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat	medv
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

506 rows × 14 columns

In [20]:
```
df.isnull().sum()
```

Out[20]:
```
crim       0
zn         0
indus      0
chas       0
nox        0
rm         0
age        0
dis        0
rad        0
tax        0
ptratio    0
b          0
lstat      0
medv       0
dtype: int64
```

In [21]:
```
#independent variable
x=df.drop("medv",axis=1)

#dependent varibale
y=df["medv"]
```

In [22]:
```
print(x.head())
```

```
      crim    zn  indus  chas    nox     rm   age     dis  rad  tax  ptratio  \
0  0.00632  18.0   2.31     0  0.538  6.575  65.2  4.0900    1  296     15.3
1  0.02731   0.0   7.07     0  0.469  6.421  78.9  4.9671    2  242     17.8
2  0.02729   0.0   7.07     0  0.469  7.185  61.1  4.9671    2  242     17.8
3  0.03237   0.0   2.18     0  0.458  6.998  45.8  6.0622    3  222     18.7
4  0.06905   0.0   2.18     0  0.458  7.147  54.2  6.0622    3  222     18.7

        b  lstat
0  396.90   4.98
1  396.90   9.14
2  392.83   4.03
3  394.63   2.94
4  396.90   5.33
```

In [23]:
```
print(y.head())
```

```
0    24.0
1    21.6
2    34.7
3    33.4
4    36.2
Name: medv, dtype: float64
```

In [12]:
```
x_train, x_test, y_train, y_test = train_test_split(x,y, test_size=0.2)
model=LinearRegression()
model.fit(x_train,y_train)
y_pred=model.predict(x_test)
```

In [24]:
```
print("Predictions:", y_pred)
```

```
Predictions: [11.10860346 23.42316451 32.31303303 19.07828003 16.62650419 25.17974127
 17.00677007 38.41397457 12.72640403 17.87475493 20.58190458 31.36467139
 30.52544024 21.38582028 33.21676321 30.63613704 20.9690682  20.54601682
 19.36170285 21.22387625  6.96914291 24.59728137 26.9015194  32.11306754
 13.95654741 28.74638869 12.534755   20.31203263 18.02573955 13.96124304
 28.43167297 19.33810431 24.6658288  27.25028369 17.03976237 17.97558202
 18.87382643 19.85764358 35.94592892 28.48160351 25.35866122 15.71269878
 19.58785508 17.90351786 20.40264979 30.5980553  19.92290687 17.2871298
 20.43992057 19.66339301 24.87565233  9.88495064 11.52202775 18.85538352
 16.18602777 14.44044641  6.18675996 20.52310153 26.84004143 11.53021291
 18.12010141 26.04627993 23.20579346 18.09593179 15.83407654 22.77861937
 23.41528104  8.20093793 19.29736033  8.88751691 19.92721064 33.24448756
 25.66524998  8.72615892 23.76429641 11.768188   32.53504321 37.86811806
 18.35149673 19.14559264 13.80495981 18.6774477  25.00850537 27.3608763
 27.91543417 17.18229908  6.74275236 32.22608839 32.70416079 17.33360574
 18.21268508 22.04899561 29.89246038 18.08198417 24.4251943   6.48204353
 13.96208566 23.37838525 22.13824146 23.17954795 28.58998713 25.49291035]
```

In [25]:
```
result = pd.DataFrame({"Actual": y_test, "Predicted": y_pred})
print(result.head())
```

```
     Actual  Predicted
399     6.3  11.108603
77     20.8  23.423165
227    31.6  32.313033
9      18.9  19.078280
123    17.3  16.626504
```

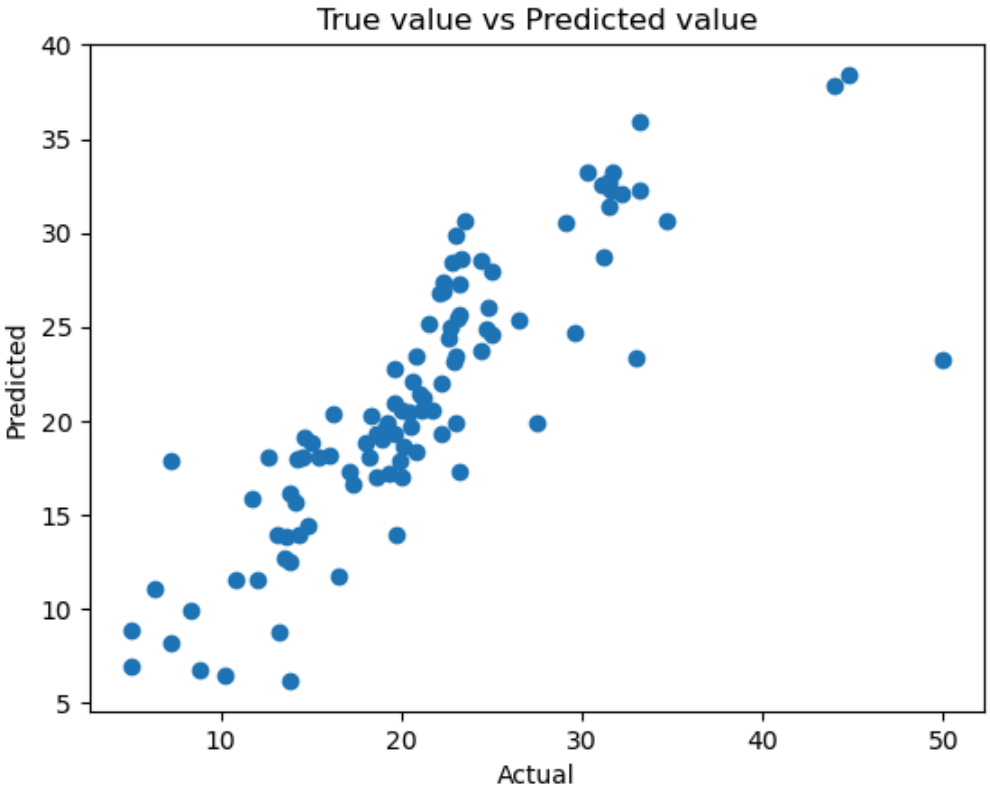
In [32]:
```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

In [33]:
```
print("MSE:", mse)
```

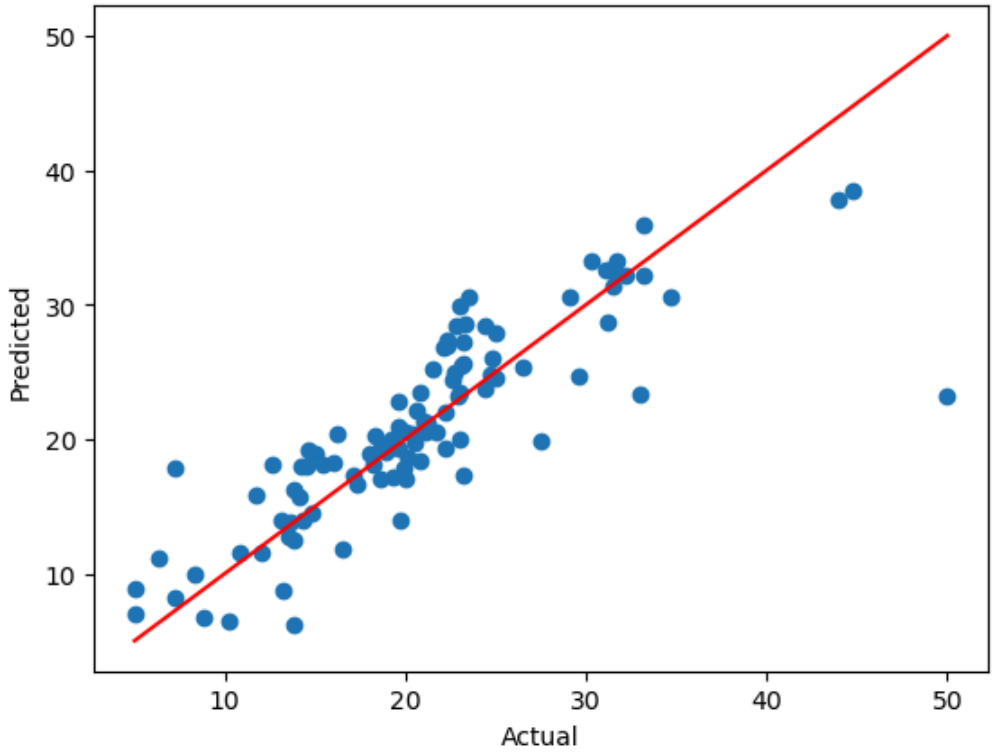
```
print("R2 Score:", r2)
```

```
MSE: 18.630337102949508
R2 Score: 0.7095798753983718
```

In [35]:
```
plt.scatter(y_test, y_pred)
plt.title("True value vs Predicted value")
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.show()
```



In [36]:
```
plt.scatter(y_test, y_pred)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         color='red')
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.show()
```



In [ ]: